DAA EXPERIMENT 4

Binary Search with Big-O Analysis

Aim

To implement Binary Search using Python and analyze its time and space complexity.

Algorithm

1. Start.
2. Read the elements of a sorted array and the search key.
3. Initialize low to 0 and high to n - 1.
4. Calculate the middle index.
5. If the middle element equals the key, display its index.
6. If the key is greater than the middle element, search the right half; otherwise, search the left half.
7. Repeat until the element is found or low becomes greater than high.
8. Stop.

Program

```
# Binary Search (input array must be sorted)

n = int(input("Enter number of elements: "))
arr = []

print("Enter elements in sorted order:")
for i in range(n):
    arr.append(int(input("Enter element: ")))

key = int(input("Enter element to search: "))

low = 0
high = n - 1

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at index:", mid)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Element not found")
```

Complexity Analysis

- Best Case: O(1)
- Average Case: O(log n)

- Worst Case: O(log n)
- Space Complexity: O(1)

Output

```
Enter number of elements: 5
Enter element: 10
Enter element: 20
Enter element: 30
Enter element: 40
Enter element: 50
Enter element to search: 40
Element found at position: 3
PS C:\Users\jithe\OneDrive\Documents\DAA - programs>
```

Result

Thus, Binary Search was successfully implemented and its time and space complexity were analyzed.